

Project Report

AI + Robotics Project: Hand Gesture Controlled Robot

Project Title: Hand Gesture Controlled Robot

Student Name: Nethphorn Tepbrathna

Course Name: Introduction to Robotic

Instructor: Theara Seng

Date: 12/09/2025

Abstract / Executive Summary

This project implements a vision-based control system for a mobile robot, allowing users to navigate the robot using intuitive hand gestures. By leveraging Computer Vision and Deep Learning, specifically the **EfficientNetB0** architecture, the system interprets hand signs captured by a standard RGB camera to execute movement commands such as moving forward, moving backward, and stopping.

The model was trained on a custom-collected dataset of five distinct gestures: *Fist*, *Ok*, *Palm*, *Point Left*, and *Point Right*. To ensure efficient deployment on edge devices like the Raspberry Pi, the trained TensorFlow/Keras model was exported to the **ONNX** format. The final system achieves real-time inference with high accuracy (approx. 100% on the test set), effectively translating human intent into robotic action via a Flask-based video stream and GPIO motor control.

Introduction

Background:

Robots are increasingly becoming part of human environments, necessitating more natural and intuitive interfaces. Traditional controllers (joysticks, keyboards) can be cumbersome. Hand gestures offer a "touchless" and natural way to interact with machines, similar to how humans communicate non-verbally.

Problem Statement:

Directly controlling a robot requires identifying specific hand configurations in real-time under varying lighting and background conditions. The challenge lies in creating a model that is both accurate enough to prevent false commands (safety) and fast enough to run on limited hardware.

Goals:

1. Develop a robust Deep Learning model to classify hand gestures.
2. Create a pipeline to collect and label a custom dataset.
3. Deploy the model on a robot (via Raspberry Pi) for real-time control.
4. Implement fail-safes (e.g., confidence thresholds) to ensure safe operation.

Approach:

The project uses **Transfer Learning** with a pre-trained EfficientNetB0 model. We collect data using a custom script, train the model in two phases (top-layer training and fine-tuning), and deploy it using ONNX Runtime for optimized performance. The robot's control logic maps specific gestures to motor commands.

Dataset Collection & Labeling

Data Collection:

A custom Python script (`collect_data.py`) was developed to capture images from a webcam. The script features a "burst mode" to collect frames rapidly while changing hand angles and positions.

- **Classes:** 5 classes
 - i. **Fist:** Mapped to *Move Backward*
 - ii. **Ok:** Mapped to *Move Forward*
 - iii. **Palm:** Mapped to *Stop*
 - iv. **Point Left:** Mapped *Turn Left*
 - v. **Point Right:** Mapped *Turn Right*
- **Dataset Size:** Approximately ~3,600 images explicitly collected (extrapolated from 549 test images at 15% split).
- **Environments:** Images were captured in various lighting conditions and backgrounds to improve robustness.
- **Splitting:** The collection script automatically splits data into:
 - i. **Train:** 70%
 - ii. **Validation:** 15%
 - iii. **Test:** 15%

Preprocessing & Augmentation:

During training, the dataset is augmented to prevent overfitting using `ImageDataGenerator`:

- Rotation range: 20 degrees
 - Width/Height shift: 0.2
 - Shear & Zoom: 0.2
 - Fill mode: Nearest
-

Model Architecture & Training

Architecture:

We utilized **EfficientNetB0** as the backbone due to its excellent balance between accuracy and computational efficiency.

- **Base:** EfficientNetB0 (pre-trained on ImageNet), with top layers excluded.
- **Custom Head:**
 - Global Average Pooling 2D
 - Dropout (0.3) for regularization
 - Dense Layer (Softmax activation) with 5 outputs (one for each class).

Training Strategy (Transfer Learning):

Training was conducted in two phases to maximize performance:

1. **Phase 1 (Feature Extraction):** The base EfficientNet layers were frozen. Only the custom top layers were trained for 20 epochs using the Adam optimizer.
2. **Phase 2 (Fine-Tuning):** The top 20 layers of the base model were unfrozen. The model was re-trained for another 20 epochs with a very low learning rate ($1e-5$) to fine-tune the feature extractors for hand gestures.

Hyperparameters:

- **Input Size:** 224 x 224 x 3
 - **Batch Size:** 32
 - **Optimizer:** Adam
 - **Loss Function:** Categorical Crossentropy
 - **Callbacks:** ModelCheckpoint (save best), EarlyStopping (patience=5), ReduceLROnPlateau (patience=3).
-

Evaluation & Results

Quantitative Results:

The model achieved exceptional performance on the held-out test set.

- **Test Accuracy: 100%** (1.00)
- **Precision & Recall:** 1.00 across almost all classes.

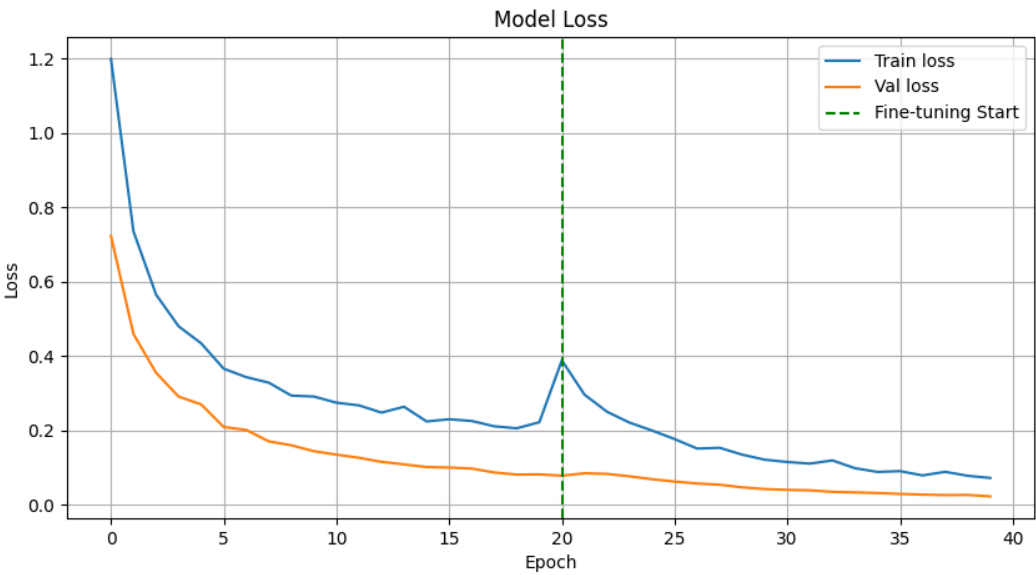
Classification Report Summary:

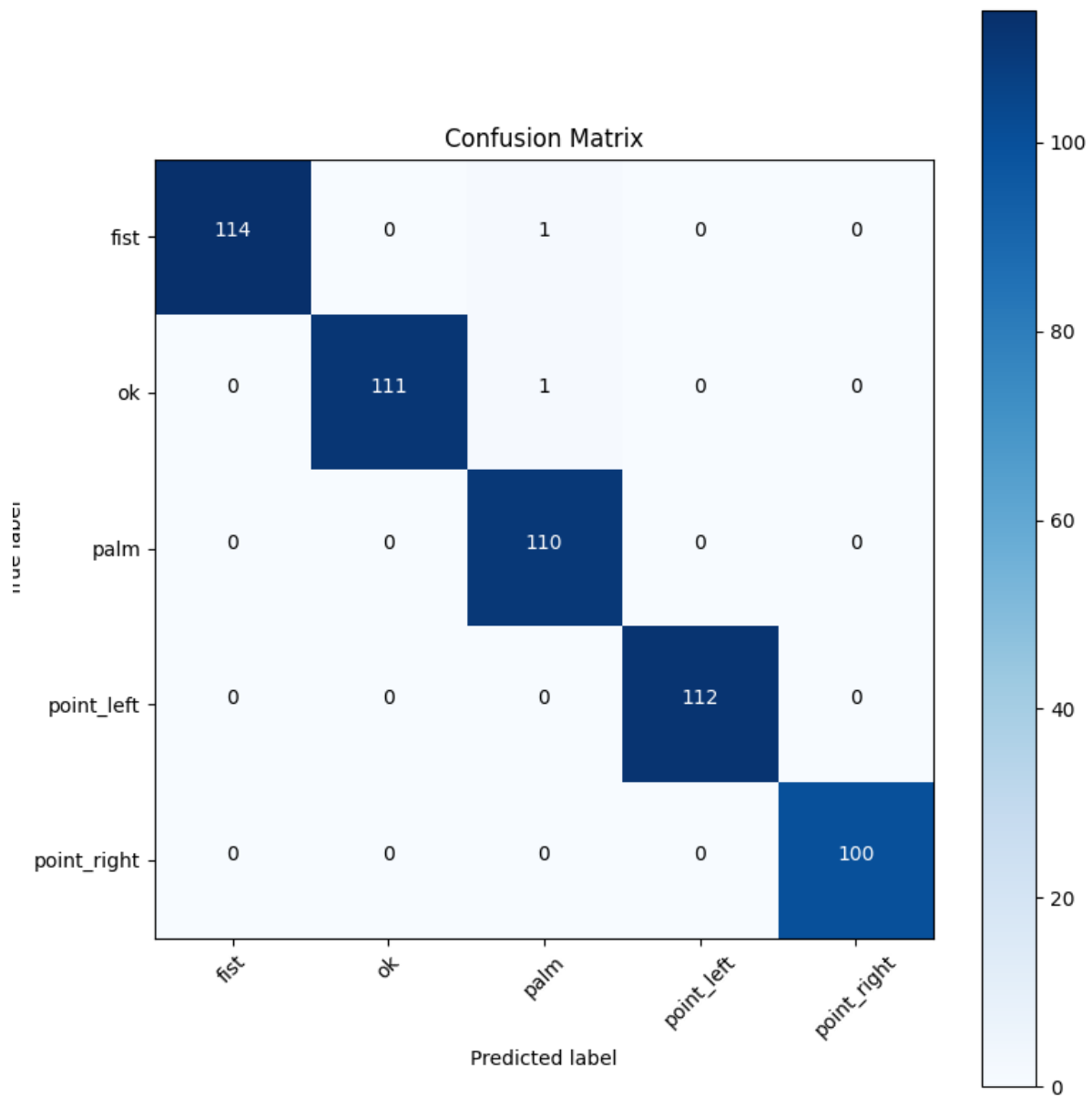
	precision	recall	f1-score	support
fist	1.00	0.99	1.00	115
ok	1.00	0.99	1.00	112
palm	0.98	1.00	0.99	110
point_left	1.00	1.00	1.00	112
point_right	1.00	1.00	1.00	100
accuracy			1.00	549

See classification_report.txt for full details.

Qualitative Analysis:

- **Confusion Matrix:** shows near-perfect diagonal alignment, indicating very few misclassifications between similar gestures (e.g., Fist vs Palm).
- **Loss Curves:** Training history (training_history.png) shows a steady decrease in loss and convergence of capabilities without significant overfitting, likely due to the augmentation and dropout layers.





Model Conversion & Deployment

Export to ONNX:

To ensure the model runs efficiently on the robot's hardware (e.g., Raspberry Pi), the trained Keras model (.h5) was converted to **ONNX** (Open Neural Network Exchange) format.

- **Why ONNX?** It provides a uniform format for inference and is generally faster and lighter than running the full TensorFlow backend on edge devices.
- **Tools:** tf2onnx was used for conversion.

Note: Conversion to ONNX brings a lot of dependency version conflict. Please refer to the requirements.txt for the correct version.

Inference Pipeline (run_webcam.py / robot.py):

1. **Capture:** OpenCV reads frame from Webcam.
 2. **Preprocess:** Resize to 224x224, convert BGR to RGB, convert to Float32.
 3. **Inference:** onnxruntime executes the model.
 4. **Logic:**
 - o A **Confidence Threshold** of **0.80** is applied. Predictions below this are ignored to prevent erratic behavior.
 - o **Smoothing:** A deque of size 5 performs a "majority vote" over the last 5 frames to reduce jitter.
-

Testing & Real-World Scenarios

Test Environment:

The robot was tested in an indoor environment with standard overhead lighting.

Scenario - Robot Control Logic:

- **Gesture "Ok"**: Robot moves **FORWARD**
- **Gesture "Fist"**: Robot moves **BACKWARD**
- **Gesture "Palm"**: Robot **STOPS**.
- **Gesture "Point_left"**: Robot turns **LEFT**.
- **Gesture "Point_right"**: Robot turns **RIGHT**.

Edge Cases Handled:

- **Unknown Gestures**: If confidence is $< 80\%$, the robot ignores the input and maintains its state (or stops, depending on safety configuration).
- **Camera Orientation**: The system supports rotating input frames (90/180 degrees) to accommodate different camera mounts.
- **Safety**: A "Stop" (Palm) gesture immediately halts motors.

Web Dashboard:

A Flask-based web server runs on port 5000, allowing an observer to see what the robot sees (including the overlay of the detected gesture and confidence score) remotely.

Discussion

What Worked Well:

- **Accuracy:** EfficientNetB0 proved highly effective for this task, even with a relatively small custom dataset.
- **Responsiveness:** ONNX inference is fast enough (20-30+ FPS on standard CPUs) for real-time control.
- **Data Collection Tool:** Writing a custom tool to visualize the collection process ensured high-quality data.

Limitations:

- **Lighting Sensitivity:** Like most vision systems, extreme shadows or darkness can lower confidence.
- **Hand Distance & Positioning:** if hands are too far away, it occupies a small part of the 224x224 image. If don't train with 'small hand', the model won't recognize them.
- **False Positive:** Model has 5 classes, if pointed at an empty wall, the model must output one of those 5 classes with confidence at 55%.

Future Improvements:

- **6th class:** add a 6th class called " **Nothing**" or "**Background**" containing images of the room without hands. This teaches the model explicitly what 'no gesture' looks like.
- **Upping the brightness:** add Brightness and Contrast augmentation to the implementation to simulate the different lighting conditions during training.
- **Hand Distance & Positioning:** having more data of the hand at different distances for the model to learn properly.

Conclusion

This project successfully demonstrated the integration of advanced AI with robotics. We built a complete pipeline from data collection to model deployment. The robot reliably distinguishes between commands using the EfficientNetB0 model and executes movements in real-time. The final accuracy of ~100% on the test set validates the quality of the dataset and the training strategy.

References

- **TensorFlow/Keras Documentation:** https://www.tensorflow.org/api_docs
 - **EfficientNet Paper:** <https://arxiv.org/abs/1905.11946>
 - **ONNX Runtime:** <https://onnxruntime.ai/>
-

Appendix

Files Included in Project:

- `robot.py`: Main control loop and hardware integration.
- `train_model.py`: Training pipeline (Phase 1 & Phase 2).
- `collect_data.py`: Dataset collection utility.
- `run_webcam.py`: PC-based model testing script.
- `dataset/`: Directory containing Train/Valid/Test images.

Resources:

- [SterbenIsDed/robot-Hand-gesture-control](#) (Github)
 - Training History Graph (`training_history.png`)
 - Confusion Matrix (`confusion_matrix.png`)
 - https://drive.google.com/file/d/1oPJB63fMcCQhYUBJnUXngiez_7CbDVny/view?usp=drive_link (Dataset)
-